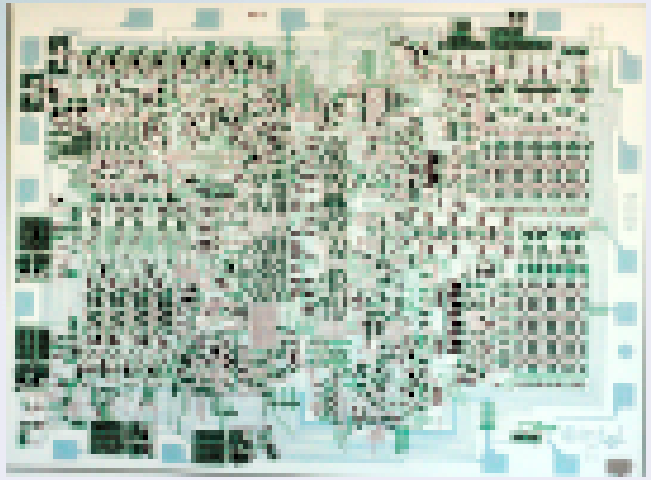

Lecture 9b: Writing Traps and Interrupts

CSCI11: **Computer Architecture and Organization**

Quick Review: TRAP Execution

```
TRAP xnn  →  push PSR, push PC  →  switch to supervisor
          →  PC ← M[x00nn]      →  handler runs
RTI       →  pop PC, pop PSR    →  back to user code
```

The **stack** is the mechanism that makes "call OS / return to me" work safely.

Today: Writing I/O Handlers

How Does a Computer Talk to Hardware?

The LC-3 uses **memory-mapped I/O** — devices appear as special memory addresses:

Strategy	How
Port-mapped I/O	Dedicated I/O instructions
Memory-mapped I/O	Devices are addresses — read/write them like memory

The LC-3 uses memory-mapped I/O. The keyboard and display each have two registers: a **status register** (is the device ready?) and a **data register** (the actual character).

Architectures That Still Use Port-Based I/O

x86 / x86-64 (Intel & AMD)

The most prominent modern holdout. The `IN` and `OUT` instructions still exist and are used for:

- Legacy PC hardware (PCI config space, RTC, PIC/APIC)
- BIOS/UEFI firmware
- Industrial PC applications
  However, even on x86, most modern drivers use memory-mapped I/O. Port I/O persists mainly for backward compatibility.

Why Modern Designs Abandoned Port-Based I/O

Nearly all modern microcontrollers — including the **AVR family** — use **memory-mapped I/O exclusively**. The reasons are compelling:

- **Simpler ISA** — no need for separate `IN`/`OUT` instructions; normal load/store instructions work on peripherals
- **C-friendly** — a memory-mapped register is just a pointer dereference (`*PORTB = 0xFF`), whereas port I/O requires inline assembly or compiler intrinsics
- **Larger address space** — MMIO scales easily; port address spaces are typically limited (x86 has only 64K port addresses)
- **Uniform memory model** — DMA controllers, caches, and buses all work with a single address space

Port-based vs. Memory-mapped

Feature	Port-Based I/O	Memory-Mapped I/O
Instruction set	Needs `IN`/`OUT`	Uses normal load/store
Address space	Separate, usually small	Shared with RAM
C access	Intrinsics/inline ASM	Simple pointer dereference
Bus design	Needs separate I/O signal line	Unified address bus
Examples	x86, z80	AVR, ARM, RISC-V, MIPS

Output

Display Device Registers

Address	Name	Role
xFE04	**DSR**	Display Status Register
xFE06	**DDR**	Display Data Register

DSR bit 15 — set to 1 when the display is *ready* to accept a character. Note this is the same as DSR being *NEGATIVE*.

DDR — write an ASCII character here and it appears on the screen.

The OUT trap handler (TRAP x21) polls DSR in a loop, then writes to DDR. Your program never touches these registers directly. **Because these addresses are in the supervisor-only region, accessing them requires a TRAP.**

Polling: Waiting for the Display

This code runs inside the OS OUT handler — in supervisor mode at an address in the OS region. It cannot run from x3000.

LDI / STI — Load/Store Indirect: the operand is an address *of* an address.

Polling: Code

```
; --- OS OUT handler (supervisor mode) ---
; Entry: R0 = ASCII character to display
; This is the core of what TRAP x21 executes

OUT_HANDLER ADD R6, R6, #-1
            STR R1, R6, #0      ; push R1 (scratch register)

POLL_DSR    LDI  R1, DSR_ADDR   ; read Display Status Register
            BRzp POLL_DSR       ; loop while bit 15 = 0 (not ready)
            STI  R0, DDR_ADDR   ; display is ready — write character

            LDR R1, R6, #0
            ADD R6, R6, #1      ; pop R1
            RTI                 ; return to user program

DSR_ADDR    .FILL xFE04
DDR_ADDR    .FILL xFE06
```

Calling the OUT Handler

From user code, you never write the polling loop. You write:

```
            .ORIG x3000
            LD  R0, CHAR        ; load ASCII value into R0
            OUT                 ; mnemonic for TRAP x21
            HALT

CHAR        .FILL x0041         ; 'A'
            .END
```

`OUT` is an assembler mnemonic for `TRAP x21`. The hardware jumps to the OS handler, which polls DSR and writes DDR, then RTI returns here. No R6 initialization needed — the TRAP switches to the supervisor stack on its own.

INPUT

Keyboard Device Registers

Address	Name	Role
xFE00	**KBSR**	Keyboard Status Register
xFE02	**KBDR**	Keyboard Data Register

KBSR bit 15 — set to 1 when a key has been pressed and a character is waiting. Note this is the same as *KBSR* being *NEGATIVE*

KBDR — read this to get the ASCII value of the character. Reading it clears KBSR.

The GETC trap handler (TRAP x20) polls *KBSR* in a loop, then reads *KBDR*. Your program never reads these registers directly.

Polling: Waiting for the Keyboard

This code runs inside the OS GETC handler — in supervisor mode. It cannot run from x3000.

No register saves needed — R0 is the output, and GETC uses no scratch registers.

Polling: Code

```
; --- OS GETC handler (supervisor mode) ---
; Exit:  R0 = ASCII value of key pressed
; This is the core of what TRAP x20 executes

GETC_HANDLER
POLL_KBSR   LDI  R0, KBSR_ADDR  ; read Keyboard Status Register
            BRzp POLL_KBSR       ; loop while bit 15 = 0 (no key yet)
            LDI  R0, KBDR_ADDR  ; key ready — read the character
            RTI                  ; return to user program (R0 = character)

KBSR_ADDR   .FILL xFE00
KBDR_ADDR   .FILL xFE02
```

Calling the GETC Handler

From user code:

```
.ORIG x3000
GETC                    ; mnemonic for TRAP x20
                        ; R0 now holds ASCII of key pressed
OUT                     ; echo it — mnemonic for TRAP x21
HALT
.END
```

`GETC` is `TRAP x20`. After it returns, R0 contains the character. The OS polling loop ran entirely in supervisor mode while your user program waited at the TRAP instruction. No R6 initialization required for either call.

Trap I/O

Three key ideas:

1. **Stack** preserves the return address (PC) and processor state (PSR)
2. **Privilege switch** gives the handler access to device registers
3. **Vector table** decouples user code from handler addresses

Ultimately, it is a *polling* technique.

And Yet: Polling Is Simple but Wasteful

```
while keyboard not ready:
    spin and do nothing

read character
```

The CPU wastes every cycle checking "has a key been pressed yet?"

For a single program, this is acceptable.
For an OS running multiple programs — it is catastrophic.

Interrupts can solve this. The keyboard notifies the CPU when it is ready, instead of the CPU constantly asking.

Traps vs. Interrupts

	Trap	Interrupt
Initiated by	User program (`TRAP xnn`)	External device (keyboard, timer)
When	Synchronous — happens at a specific instruction	Asynchronous — can happen any time
Purpose	Request an OS service	Notify CPU that a device needs attention
Mechanism	Same: push PSR + PC, switch to supervisor, load new PC	

Both use the stack and PSR in exactly the same way. The difference is *who initiates* the context switch.

Interrupts: The Problem with Polling

The GETC handler polls in a tight loop (supervisor mode):

```
; --- Inside the OS GETC handler (supervisor mode) ---
POLL_KBSR   LDI  R0, KBSR_ADDR  ; read Keyboard Status Register
            BRzp POLL_KBSR       ; spin here until key pressed
            LDI  R0, KBDR_ADDR  ; got the key
            RTI
```

Polling

For a single program, polling is acceptable. When a program is interacting with a user, polling is typically, fine.

For an OS running multiple programs — it is catastrophic. The CPU is locked inside this handler, spinning, while every other program starves.

Interrupt-driven I/O lets the CPU do other work and get notified when the device is ready.

How an Interrupt Works

1. Device raises an interrupt signal and asserts a **priority level** (0–7)
2. CPU finishes the *current* instruction
3. If device priority > current CPU priority (PSR bits 10–8):
    - Push PSR and PC onto the **supervisor stack**
    - Set PSR priority = device priority, set supervisor mode
    - Load PC from **interrupt vector table** (x0100–x01FF)
4. Interrupt handler runs
5. `RTI` restores PSR (and old priority) and PC

The stack mechanism is *identical* to a TRAP — only the trigger differs.

Priority and Preemption

```
Priority 7 — highest (hardware clock)
Priority 6
Priority 5
Priority 4
Priority 3
Priority 2
Priority 1
Priority 0 — user programs
```

A priority 5 interrupt *cannot* be interrupted by a priority 3 device.
A priority 5 interrupt *can* be interrupted by a priority 7 device.

Each time a handler runs, its priority is stored in the PSR — so the stack captures the full nesting history.

Nested Interrupts on the Stack

```
Running user code (priority 0)
   ↓  priority 3 interrupt arrives
Push PSR(0), PC → run handler 3
   ↓  priority 6 interrupt arrives
Push PSR(3), PC → run handler 6
   ↓  handler 6 finishes
Pop PSR(3), PC  → resume handler 3
   ↓  handler 3 finishes
Pop PSR(0), PC  → resume user code
```

The stack automatically unwinds the correct context at every level.

The Supervisor Stack is Sacred

Never let your code corrupt the supervisor stack.

- The OS relies on exactly the PSR and PC it pushed being there when `RTI` executes
- Extra pushes without matching pops → RTI returns to garbage
- Extra pops → RTI restores a user value as a PC

In your trap handlers: **count your pushes and pops**. They must balance perfectly.

```
; Mirror your saves
            ADD R6, R6, #-1
            STR R3, R6, #0      ; push R3
            ADD R6, R6, #-1
            STR R2, R6, #0      ; push R2
            ADD R6, R6, #-1
            STR R1, R6, #0      ; push R1
            ; ...
            LDR R1, R6, #0
            ADD R6, R6, #1      ; pop R1  (reverse order!)
            LDR R2, R6, #0
            ADD R6, R6, #1      ; pop R2
            LDR R3, R6, #0
            ADD R6, R6, #1      ; pop R3
            RTI
```

Concept	Key Point
Memory-mapped I/O	Devices appear as addresses xFE00–xFFFF
Polling	Spin until status bit = 1, then read/write data register
TRAP	Synchronous OS call: push PSR+PC, switch mode, jump via vector
Interrupt	Asynchronous device event: same mechanism, triggered by hardware
Stack role	Saves return address (PC) and processor state (PSR) at every level
RTI	Restores PSR and PC; must see exactly what TRAP/interrupt pushed
Register discipline	Handler must save and restore every register it uses except agreed registers (ex: *R0*)

Next Week

- **Test 2** (Week 10) covers Units 6–9
- Topics: LC3 programming, stacks, subroutines, I/O, traps